Computer Networks 278 (2026) 112073

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

MAE: Collaborative inference acceleration with efficient DNN partitioning and resource allocation in resource-constrained edge computing

Juan Fang, Yaxin An, Yaqi Liu, Ziyi Teng, Xiaoning Zhai, Heng Tang, Huijie Chen *

College of Computer Science, Beijing University of Technology, Beijing, 100124, China

ARTICLE INFO

Keywords:
Collaborative inference
DNN partitioning
Resource allocation
Edge computing

ABSTRACT

Mobile Edge Computing (MEC) serves as a critical architecture that empowers resource-constrained mobile terminals to provide neural network driven edge intelligence services. Its core challenge centers on addressing the efficiency bottleneck of Deep Neural Network (DNN) inference in edge-device collaboration scenarios. Although existing DNN partitioning techniques facilitate the cooperative deployment of computation-intensive DNNs between User Equipment (UEs) and edge servers (ES), these techniques impose a theoretical upper bound on optimal inference latency since they fundamentally fail to modify the DNN structure. Inspired by the widely adopted Mixture of Experts (MoE) paradigm, we introduce the Mixture of Adaptive Experts (MAE) framework, which is specifically designed for heterogeneous collaborative DNN inference in resource-constrained edge computing environments. Specifically, MAE reconfigures convolutional channels as specialized "experts"; that is, only a sparse subset of these convolutional channels are activated during each inference process. This design significantly reduces intermediate feature transmission overhead while preserving model accuracy. Furthermore, MAE facilitates efficient model partitioning and the subsequent selection of expert models. Notably, MAE fundamentally transforms the computationally intractable resource scheduling challenge into an optimization problem defined by a monotonically decreasing function. To address this optimization problem, we propose an algorithm called MAE resource allocation which is capable of achieving the optimal solution in logarithmic time. Large-scale experiments on real-world heterogeneous DNN tasks showcase that our solution significantly improves overall system performance while guaranteeing inference accuracy, thereby conclusively validating its effectiveness and efficiency in practical edge computing environments.

1. Introduction

The rapid advancement of Internet of Things (IoT) technology has driven an explosion in data generated by User Equipment (UEs) (e.g., smartphones, wearables, sensors), creating an urgent demand for real-time processing by Deep Learning (DL) applications [1]. However, these devices are inherently constrained by limited computational resources, storage capacity, and battery life, rendering them incapable of independently executing complex DL inference tasks [2]. Against this backdrop, Edge Intelligence (EI) has emerged as a critical solution: it enables DL inference capabilities at the network edge, thereby facilitating low-latency and high-efficiency processing in close proximity to data sources [3].

By deploying GPU-accelerated computational resources at the network edge, edge servers (ES) allow UEs to offload computation-intensive DL inference tasks for remote execution, with the processed results then returned to the UEs [4,5]. As the primary computational workload in such DL tasks, Deep Neural Networks (DNNs) confront critical challenges in real-time execution at the network edge stemming from both their inherent computational complexity and resource constraints on ES. To mitigate these challenges, two dominant paradigms have thus been widely adopted in edge-assisted DNN inference: Full Offloading to ES and Partition-based Collaborative Inference.

Full offloading to ES addresses the inherent computational complexity of DNNs by offloading and executing entire models on powerful GPU-equipped ES. By leveraging edge-side parallel computing capabilities, this paradigm effectively accelerates the inference of computationally intensive DNN layers [6]. However, in scenarios where GPU resources are scarce, newly incoming inference tasks are forced to queue for available computing resources, resulting in considerable scheduling delays during peak load periods [7]. This queuing effect not only prolongs

* Corresponding author.
E-mail addresses: fangjuan@bjut.edu.cn (J. Fang), an-ya-xin@emails.bjut.edu.cn (Y. An), liuyq617@emails.bjut.edu.cn (Y. Liu), tengziyi@emails.bjut.edu.cn (Z. Teng), ZXNTT@emails.bjut.edu.cn (X. Zhai), tangheng@emails.bjut.edu.cn (H. Tang), chenhuijie@bjut.edu.cn (H. Chen).

https://doi.org/10.1016/j.comnet.2026.112073
Received 22 October 2025; Received in revised form 28 December 2025; Accepted 29 January 2026
Available online 3 February 2026

end-to-end latency, but also leads to the underutilization of idle computational resources on UEs. Furthermore, the transmission of unprocessed raw input data (e.g., high-resolution medical images or LiDAR point cloud scans) often becomes the dominant contributor to end-to-end latency. Under unstable network conditions, this resulting communication overhead frequently emerges as the primary factor driving overall end-to-end delay [8].

Meanwhile, existing works have leveraged pruning and distillation techniques to reduce the computational complexity of DNN models [9]. For instance, Bailey et al. proposed DNNShifter [10], a framework that combines the advantages of unstructured and structured pruning to enable the rapid generation of task-adapted lightweight DNN variants. This design allows the model to adapt to the resource constraints of edge devices while supporting efficient model switching. Majid et al. introduced a feature-based knowledge distillation method using Tucker decomposition [11], a technique that decomposes the high-dimensional feature maps of the teacher model into core tensors and employs an adapter function to align spatial dimensions. This approach thereby enables the student model to attain accuracy comparable to that of the teacher model, while significantly reducing both parameter count and computational cost.

Partition-based collaborative inference fundamentally redefines edge-device collaboration by leveraging the sequential layer architecture of DNNs, where each layer's output serves as the next layer's input. This approach delivers significant advantages over full offloading. Primarily, it eliminates the transmission bottleneck by replacing high-dimensional raw input data with progressively compressed intermediate features, thereby drastically reducing communication overhead and latency [12]. Furthermore, it optimizes computational efficiency through strategic layer allocation: compute-intensive layers are offloaded to GPU-equipped ES, while lightweight layers are retained on User Equipment (UEs). This allocation strategy harnesses underutilized terminal resources and alleviates ES congestion, ultimately reducing queuing delays [13,14].

Subsequent studies have extended this framework to multi-task scenarios [13–17], among which [13] proposes a layer-level scheduling algorithm that is theoretically optimal for tree-structured computation graphs. Despite these advancements, scaling partitioned inference to multi-DNN workloads remains fundamentally challenged by resource contention in capacity-constrained ES. Efficient resource allocation is thus crucial for balancing heterogeneous DNN execution across shared edge infrastructure. However, current approaches suffer from two compounding limitations: (1) Partition configurations grow exponentially due to the architectural heterogeneity of DNNs, rendering exact optimization computationally intractable; (2) State-of-the-art DNNs exhibit inherent computational intensity, which imposes an inherent performance ceiling even under optimal partitioning.

To address these limitations, we leverage the critical insight that end-to-end inference latency is predominantly dictated by the volume of intermediate feature transmission [12]. Consequently, minimizing this transmission data is paramount to reducing overall delay. To this end, we propose the Mixture of Adaptive Experts (MAE), a specialized Mixture of Experts (MoE) adaptation tailored for resource-constrained edge environments. Diverging from conventional edge MoE solutions that treat experts as independent models or server-side entities, MAE redefines experts as sparse convolutional channel subsets within the DNN. By employing structured sparsity, MAE directly mitigates the communication bottleneck in collaborative edge inference rather than relying on peripheral optimizations such as task scheduling or long-term learning. This design drastically reduces communication overhead while maintaining inference accuracy, providing a robust adaptation for bandwidth-limited edge networks.

Building upon this architectural innovation, the MAE framework facilitates efficient collaborative DNN inference in edges. As illustrated in Fig. 1, the DNN model is divided into two parts at a predefined partition point. Subsequently, the MAE framework converts the neural layer in DNN preceding this partition point into an MoE layer. Within the MoE layer, different activations of convolutional channels are mapped to distinct expert models. To select appropriate expert models, the MAE framework activates the minimum number of convolutional channels in the MoE layer while ensuring inference accuracy is preserved. Additionally, this framework efficiently determines the optimal partition point for expert models in edge-device collaborative inference scenarios. Furthermore, computational resources of ES that provide computing services are quantified as Minimum Computational Resource Units (MCRUs). These MCRUs are dynamically allocated in accordance with the actual computational demands of heterogeneous expert models, thereby significantly reducing the maximum inference latency of heterogeneous DNNs.

To comprehensively evaluate the efficacy and robustness of MAE, extensive experiments were conducted in an ES environment with heterogeneous DNN workloads. Experimental results demonstrate that this innovative approach breaks through the computational performance ceiling via sparse expert activation and eliminates partitioning complexity through architectural sparsity. The principal contributions of this work are articulated as follows:

Expert Model Designed for EI: We architect specialized MoE experts for collaborative edge intelligence via the activation of disparate channels, demonstrating that such designs simultaneously surpass the accuracy of policy models and extend the theoretical latency-accuracy frontier, a critical advancement for resource-constrained multi-task environments.

Framework Innovation: We propose Mixture of Adaptive Experts (MAE), a novel architecture that dynamically allocates constrained ES resources in response to fluctuating computational capacities and mobile task demands. MAE breaks through the algorithmic time complexity barrier of existing intelligent partition decision-making schemes while substantially reducing maximum inference latency without compromising model accuracy.

Prototyping and Experimental Validation: We implement an end-to-end hardware-software prototype of the MAE framework, deploying diverse heterogeneous DNNs across UEs. Rigorous experiments validate MAE under systematically varied workload conditions, including scaling heterogeneous task volumes and modulating task-type composition ratios. These tests conclusively demonstrate MAE's operational stability and robustness to workload dynamics. Standard benchmark evaluations [18] further confirm MAE's superiority over state-of-the-art baselines, achieving significant reductions in inference latency and notable improvements in resource utilization efficiency.

The remainder of this paper is organized as follows. Section 2 reviews related works and highlights the innovation of the proposed framework. Sections 3 and 4 formulate the resource-constrained optimization problem and the MAE framework's core algorithm with rigorous theoretical analysis and proofs respectively. Section 5 evaluates system performance through extensive experiments on a physical edge platform. Section 6 concludes this paper and outlines future work.

2. Related work

2.1. Edge-device collaborative DNN inference acceleration

The sequential layer-wise structure of DNNs has motivated extensive research on device-edge collaborative inference, primarily through layer partitioning. Kang et al. [12] introduced the Neurosurgeon framework to identify optimal partition points via exhaustive search, reducing inference latency in single-user scenarios. However, it lacks scalability for multi-user environments. Duan et al. [13] extended this by jointly optimizing multiple concurrent DNN tasks, enabling co-scheduling of heterogeneous workloads. Yet, under resource-constrained edge conditions, performance degrades as user numbers increase. This scalability limitation is common across mainstream DNN partitioning techniques [14,15,17,19]. Early-exit mechanisms, surveyed by Haseena et al. [17], partially alleviate edge load but cannot ensure real-time

inference under strict resource constraints. Tang et al. [20] proposed iterative alternating optimization (IAO) for intelligent partitioning and resource allocation, though its high polynomial complexity increases decision latency. Similarly, Fan et al. [21] combined DNN partitioning with joint communication and computation resource allocation, but their nonlinear algorithm incurs substantial overhead.

Overall, existing partitioning-based methods require exhaustive layer-wise traversal with time complexity $O(n)$, where n is the number of layers. In contrast, our approach reduces partitioning complexity to $O(1)$ while raising the computational ceiling for inference performance.

2.2. Resource optimization and collaborative edge inference

Recent studies leverage machine learning technologies for resource allocation across networks, data centers, and edge environments. In network optimization, IADE [22] accelerates convergence in 6G networks, SA-MARL [23] combines multi-agent reinforcement learning with simulated annealing for latency, energy, and load optimization. AFED-EF [24] adapts VM allocation to handle fluctuating workloads. Edge-focused solutions include ISC-QL [25] for optimal server placement, ECMS [26] for energy prediction, and IECL [27] for real-time power and resource estimation. In communication, ANNS [28] addresses IRS-aided optimization, SGPL [29] enables secure collaborative communication, and RDRL [30] improves dynamic spectrum access in cognitive radio networks.

These works demonstrate learning's potential for system level resource management in heterogeneous, dynamic edge systems. However, they generally overlook model-level communication bottlenecks and intermediate feature transmission in multi-DNN collaborative inference.

2.3. MoE-based edge inference

Mixture-of-Experts (MoE) architectures have emerged as a key paradigm for efficient DNN inference. DeepSeek [31] employs sparse gating to activate relevant sub-networks per input, improving efficiency. Recent adaptations target resource-constrained edge environments. Kong et al. [32] propose PC-MoE, using temporal locality and expert exchangeability to maintain a compact "Parameter Committee" with minimal accuracy loss. Wang et al. [33] leverage mobile-edge networks with DRL-based controllers for selective offloading of MoE subtasks, optimizing communication and computation jointly. Li et al. [34] introduce adaptive gating for continual learning in MEC, minimizing expert usage while bounding long-term generalization error.

Unlike prior work that treats experts as offloadable units, our Mixture of Adaptive Experts (MAE) framework reconstructs convolutional channels into sparsely activated experts, substantially reducing intermediate feature transmission while preserving accuracy. MAE directly addresses model-internal communication bottlenecks, enabling efficient multi-DNN collaborative inference under limited bandwidth and computation edge constraints.

3. System modeling and problem formulation

We consider a set $\mathcal{N}$ of n heterogeneous UEs to perform DNN inference tasks with the assistance of a resource-constrained ES.

3.1. Distributed deployment of expert model

DNNs are typically composed of sequentially connected neural layers, such as convolutional, pooling, and fully-connected layers. These layers exhibit strict sequential dependencies the output of each preceding layer serves as input to its successor. This hierarchical organization enables partitioning at any inter-layer boundary without altering computational integrity. During DNN inference initiation at a user terminal, the UE first determines which experts to activate, where each expert corresponds to a specialized convolutional channel within the MoE layer. Crucially, different activation patterns yield distinct expert model by combinatorially aggregating selected channels. As illustrated in Fig. 1, the expert model is partitioned at the splitting point into a expert header model and a expert tail model, which are deployed on the UEs and the ES, respectively. Specifically, **Expert Header Model** includes all layers from input to the partitioned MoE layer (inclusive), executed on UEs. **Expert Tail Model** comprises layers subsequent to the partition MoE layer (exclusive), executed on ES.

After performing expert header model inference on UEs, intermediate feature data is transmitted to ES via wireless networks. ES then executes expert tail model inference and returns final results to UEs. Specifically, during the process of performing an inference task of type k, UE i selects expert model $e_k^i \in \mathcal{E}_k$ and partition point $s_k^i \in \mathcal{S}_k$ based on input characteristics. The header compute load $X(s_k^i, e_k^i)$ consumes local resources, intermediate data $M(s_k^i, e_k^i)$ transmits via bandwidth B_i, and tail compute load $Y(s_k^i, e_k^i)$ executes on ES using allocated Minimum Computational Resource Unit (MCRUs) f_i (yielding processing power $p_i = f_i \cdot \alpha$). Operating under resource-constrained ES provisioning multiple heterogeneous DNN inference, the system optimizes to minimize the maximum end-to-end latency D, which aggregates UEs computation, data transmission, and edge computation delays. We summarize the symbols used in the system model in Table 1.

3.2. DNN inference latency model

The end-to-end latency of DNN inference primarily comprises three fundamental components: (a) the execution time of the header model on UEs, (b) the transmission latency of intermediate data generated by the header model, and (c) the computation time of the tail expert model on ES.

Execution Time of the Header Model on UEs (t_{mob}). Let $X(s_k^i, e_k^i)$ denote the computational load (FLOPs) for executing the header model of task type k from the input layer through the partition layer s_k^i using expert model e_k^i. C_i denotes total computational capacity of UE i (FLOPs/s). Thus, t_{mob} is given by $t_{\text{mob}} = \frac{X(s_k^i, e_k^i)}{C_i}$.

Intermediate Data Transmission Latency (t_{trans}). Let $M(s_k^i, e_k^i)$ denote the size of the intermediate feature tensor (bits) generated at partition layer s_k^i using expert model e_k^i for task type k. The propagation delay incurred during intermediate data transfers is explicitly incorporated into the temporal model. Let B_i denote uplink bandwidth allocated to UE i (bps), t_{trans} is given by $t_{\text{trans}} = \frac{M(s_k^i, e_k^i)}{B_i}$.

Edge Computation Latency (t_{edge}). Let $Y(s_k^i, e_k^i)$ denote the computational load (in Floating Point Operations, FLOPs) for executing the expert tail model of task type k from the layer immediately following the partition layer s_k^i through the output layer using expert model e_k^i. Let p_i denote the computational capacity allocated to UE i (in FLOPs/s), where $p_i = f_i \cdot \alpha$. τ_{down} is the result download delay. The edge computation latency is given by:

$$t_{\text{edge}} = \underbrace{\frac{Y(s_k^i, e_k^i)}{p_i}}_{\text{computation}} + \underbrace{\tau_{\text{down}}}_{\text{download}} \tag{1}$$

The end-to-end latency is formally defined as the additive composition of these interdependent components:

$$t(s_k^i, e_k^i, f_i) = \frac{X(s_k^i, e_k^i)}{C_i} + \frac{M(s_k^i, e_k^i)}{B_i} + \frac{Y(s_k^i, e_k^i)}{p_i} + \tau_{\text{down}} \tag{2}$$

Notably, latency estimation methods relying solely on theoretical computational capacity suffer from fundamental limitations particularly in batch processing scenarios. Our experimental analysis reveals that the memory-bound nature of DNN inference tasks imposes increasingly severe performance constraints as batch sizes grow, with data access bottlenecks limiting achievable performance to merely 23.7% of the theoretical FLOPs ceiling. As comprehensively demonstrated in Fig. 2 (via

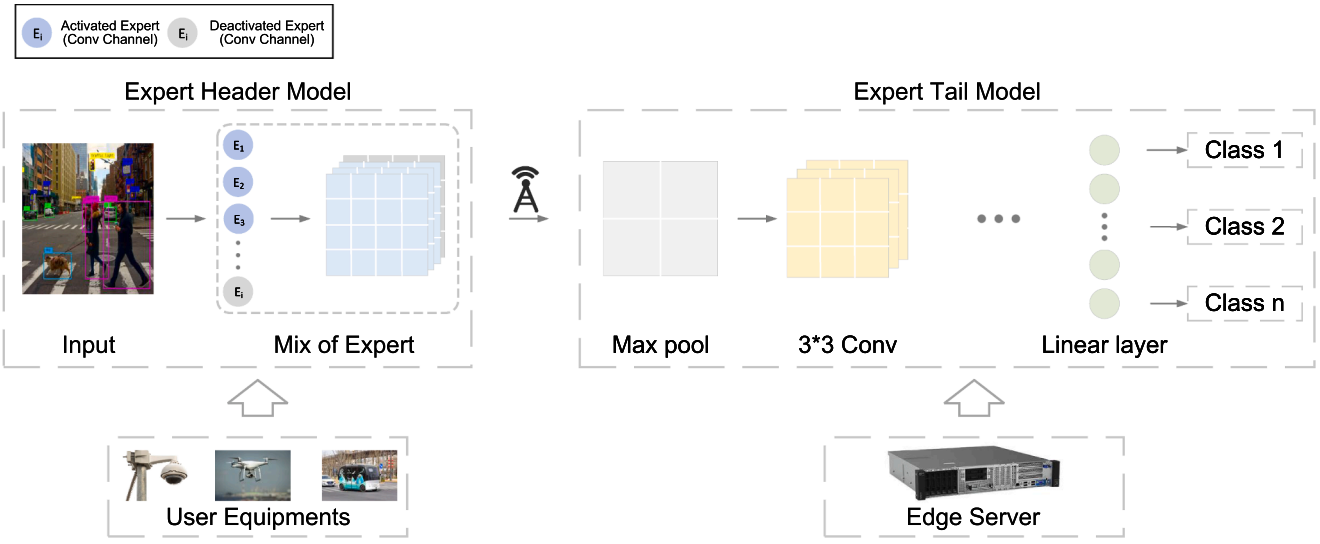


Fig. 1. MoE structured Expert Model deployed in a distributed edge-device framework.

Table 1
System Model Parameters and Definitions.

Symbol	Definition
$\mathcal{K}$	Set of inference task types (e.g., $\mathcal{K} = \{\text{AlexNet}, \text{VGG16}\}$)
$\lvert\mathcal{K}\rvert$	Number of distinct task types
$\mathcal{N}$	Set of all user equipment (UEs)
$\lvert\mathcal{N}\rvert$	Number of UEs
$\mathcal{N}_k$	Set of UEs performing task type k ($\mathcal{N}_k \subseteq \mathcal{N}$)
n_k	Number of users for task type k ($n_k = \lvert\mathcal{N}_k\rvert$)
S_k	Set of feasible partitioning points for task k
s_k^i	Selected partitioning point for task type k by UE i ($s_k^i \in S_k$)
$\mathcal{E}_k$	Set of expert models for task type k (differing in channel count)
e_k^i	Selected expert model for task type k by UE i ($e_k^i \in \mathcal{E}_k$)
$X(s_k^i, e_k^i)$	Computation load on UE side for task type k at partition s_k^i with expert model e_k^i (FLOPs)
$Y(s_k^i, e_k^i)$	Computation load on ES for task type k at partition s_k^i with expert model e_k^i (FLOPs)
$M(s_k^i, e_k^i)$	Intermediate data size transmitted for task type k at partition s_k^i with expert e_k^i (bits)
B_i	Uplink bandwidth allocated to UE i (bps)
C_i	Total computational capacity of UE i (FLOPs/s)
α	Minimum Computational Resource Unit (MCRU) size (FLOPs/s)
β	Total number of available MCRUs ($\beta = \lfloor C/\alpha \rfloor$)
f_i	Number of MCRUs of ES allocated to UE i ($f_i \in \mathbb{Z}^+$, $\sum f_i \leq \beta$)
p_i	Computational power of ES allocated to UE i: $p_i = f_i \cdot \alpha$ (FLOPs/s)
D	Maximum end-to-end latency across all UEs (s)
ϵ	Precision tolerance for optimization algorithm
D^*	Optimized maximum end-to-end latency achieved by Algorithm 1
D^{opt}	Theoretical optimal maximum end-to-end latency
η	Containerization overhead factor ($\eta \leq 1.05$)
$\delta_k^{\text{mob}}, \delta_k^{\text{trans}}$	Measurement errors for UEs computation and transmission time of task type k
$\tau_k^{\text{mob}}, \tau_k^{\text{trans}}, \gamma_k$	Average UEs computation time, transmission time, and edge load for task type k

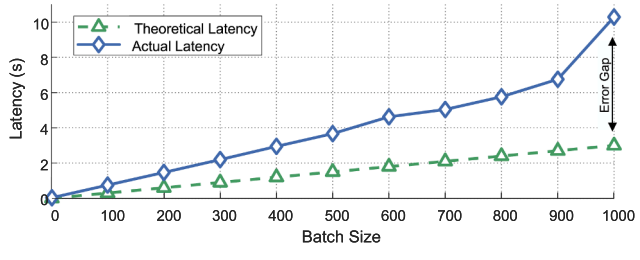


Fig. 2. Actual vs. Theoretical Inference Latency Across Different Batch Sizes.

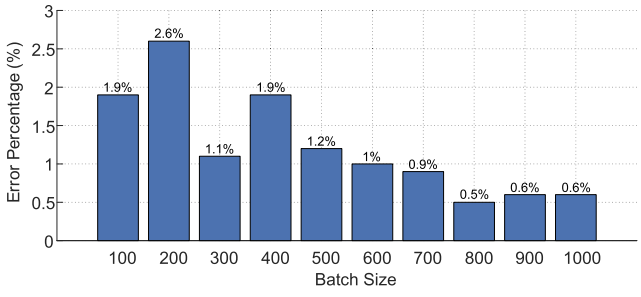


Fig. 3. The Error of the proposed Latency Prediction estimation method.

benchmarking on an Intel Core i5-9300H testbed), the divergence between theoretical latency projections and empirical measurements expands substantially with increasing batch, exposing critical limitations of existing analytical models.

For large-batch operations, persistent discrepancies between theory and practice arise from oversimplified parallelism assumptions and unaccounted for memory contention dynamics. Under small-batch

regimes, theoretical predictions exhibit significant inaccuracies, attributed to unmodeled scheduling overhead and hardware warm-up effects. Notably, there is a particularly pronounced inflection point beyond a batch size of 700, where memory bandwidth saturation occurs. These systematic underestimations stem from three primary factors: fixed framework overheads (30 ms), sublinear scaling of memory bandwidth utilization, and cache efficiency degradation at scale. Furthermore, while existing approaches [35] leverage empirical models for latency prediction, these methods exhibit strong device dependency and lack cross-platform generalizability.

To address these multidimensional prediction inaccuracies, we propose a simple yet highly effective transferable calibration methodology for accurate inference latency prediction of heterogeneous expert models in dynamic edge environments. The core of this method lies in constructing a real-time monitoring and adaptive data collection mechanism tailored to network fluctuations. Specifically, this mechanism continuously monitors key network status parameters (e.g., bandwidth, packet loss rate, and delay jitter) at ES. Once network fluctuations across consecutive time slots exceed a preset threshold (2%), the UEs automatically triggers an inference request and collects the end-to-end inference latency of each expert model under the current network state. After multiple active probing cycles, an empirical dataset of inference latency for heterogeneous expert models under diverse network conditions is systematically established providing a highly reliable basis for real-time inference latency prediction. As observed in Fig. 3, the error between the predicted and actual inference latency of expert models does not exceed 3%, with a mean error of 1.23%, which mean that the proposed inference latency prediction method closely aligns with the actual inference latency of expert models.

3.3. Problem formulation

Building upon the system model and the latency decomposition, we formulate a resource-constrained optimization problem that aims to minimize the worst-case end-to-end inference latency among all user equipments (UEs). Specifically, we adopt a min-max objective to ensure quality-of-service (QoS) fairness across heterogeneous tasks by minimizing the maximum latency experienced by any UE in the system. Let D denote this maximum end-to-end latency, which is defined as

$$D = \max_{i\in\mathcal{N},k\in\mathcal{K}} t(s_k^i, e_k^i, f_i) \tag{3}$$

The optimization objective is therefore to minimize D. For each UE $i \in \mathcal{N}$ executing task type $k \in \mathcal{K}$, we define three decision dimensions: Partition point selection ($s_k^i \in S_k$), expert model selection ($e_k^i \in \mathcal{E}_k$), and Resource allocation ($f_i \in \mathbb{Z}^+$). In addition, the first two of these decisions are discrete choices, while the last is an integer-valued allocation of MCRUs. Moreover, the end-to-end latency of task type k for each UE i must not exceed the maximum latency D_γ:

$$t(s_k^i, e_k^i, f_i) \le D_\gamma, \quad \forall i \in \mathcal{N}, \forall k \in \mathcal{K} \tag{4}$$

The optimization is subject to the following fundamental system constraints:

1. *Edge resource capacity constraint*: The total allocated MCRUs cannot exceed server capacity

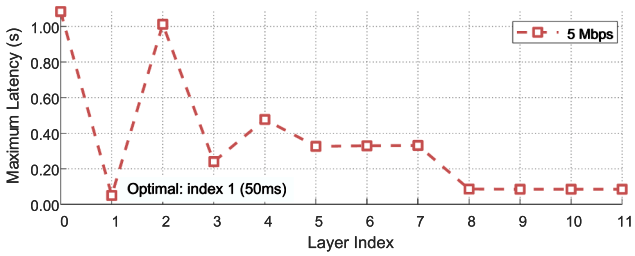
$$\sum_{i=1}^{|\mathcal{N}|} f_i \le \beta \tag{5}$$

2. *Integer resource allocation constraint*: Resource allocation is quantized in MCRU units

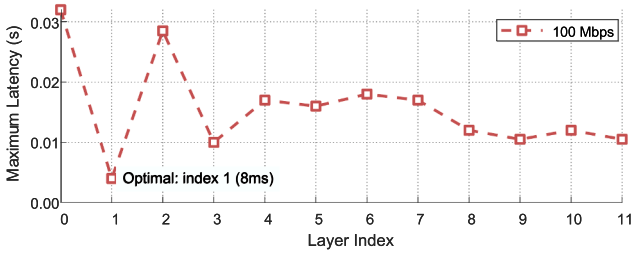
$$f_i \in \mathbb{Z}^+, \quad \forall i \in \mathcal{N} \tag{6}$$

3. *Partition feasibility constraint*: Partition points must be valid for each task type

$$s_k^i \in S_k, \quad \forall i \in \mathcal{N}_k, \forall k \in \mathcal{K} \tag{7}$$



(a) Low Bandwidth



(b) High Bandwidth

Fig. 4. Analysis of Optimal DNN Partition Point under Low and High Bandwidth.

4. *Expert model selection constraint*: expert models must be compatible with task type

$$e_k^i \in \mathcal{E}_k, \quad \forall i \in \mathcal{N}_k, \forall k \in \mathcal{K} \tag{8}$$

Thus, the joint optimization problem for MAE framework is formally stated as:

$$\mathbf{P1}: \min_{s_k^i, e_k^i, f_i} D \qquad \text{s.t. } (5)-(8) \tag{9}$$

The formulated problem exhibits three fundamental complexities:

1. *Mixed-integer nonlinear programming (MINLP)*: Combines discrete decisions (s_k^i, e_k^i) with integer allocation (f_i) and nonlinear latency constraints.
2. *High-dimensional solution space*: The search space cardinality is:

$$|\Omega| = \prod_{k\in\mathcal{K}} \left(|S_k| \times |\mathcal{E}_k|\right)^{n_k} \times \binom{\beta + |\mathcal{N}| - 1}{|\mathcal{N}| - 1} \tag{10}$$

where the binomial term represents the MCRU allocation combinations.
3. *Task-resource coupling*: Resource allocations are globally coupled through the constraint, while expert and partition selections locally affect computational and communication loads.

A key insight underpins an efficient solution: For fixed configurations of (s_k^i, e_k^i), the minimum feasible D decreases monotonically as β increases. This monotonicity property provides the theoretical basis for our proposed $O(n)$ solution, detailed in Section 4. While joint optimization across multiple resource dimensions introduces combinatorial complexity, the core monotonicity principle established herein offers a scalable theoretical foundation for future integrated resource management schemes. This focus is further validated by the operational reality of edge systems: the reconfiguration timescales of non-computational resources are typically orders of magnitude slower than those of computational resource allocation.

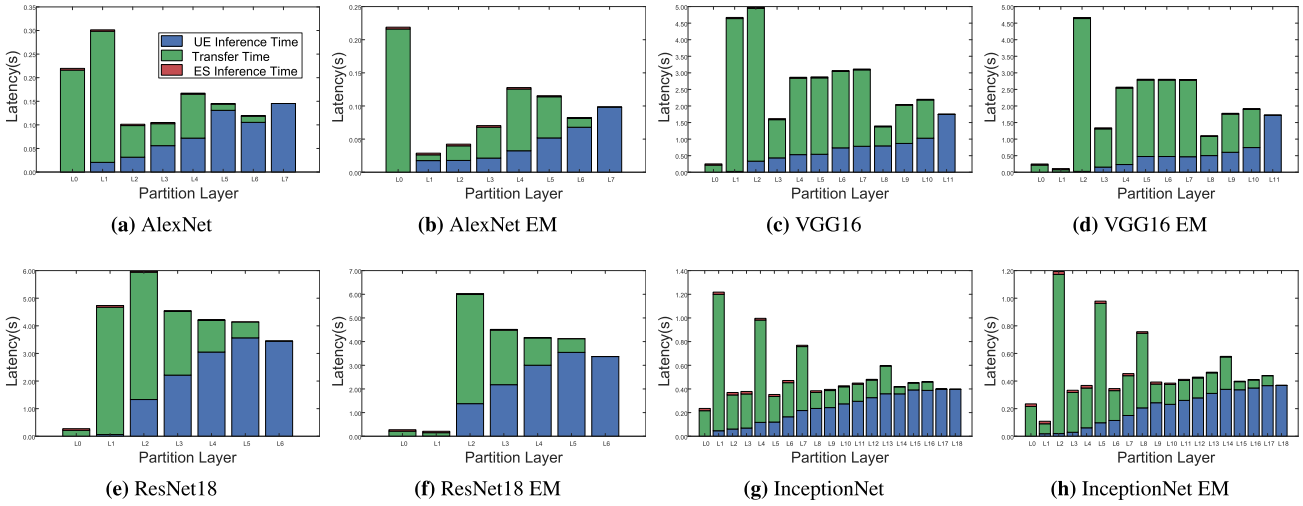


Fig. 5. Inference Latency Comparison Across Different Partitioning Strategies for Heterogeneous DNNs and Their Expert Model (EM).

4. The mae framework: an o(k)-complexity optimization algorithm

4.1. Core architectural innovations

The MAE framework revolutionizes edge-device collaborative inference through two fundamental innovations that collectively achieve $O(k)$ time complexity:

1. Fixed Optimal Partition Point: The computationally intensive portions of a DNN are typically executed on an ES. To empirically validate the optimal partition point of expert networks, an extensive set of experiments was conducted. We first examined the optimal partition point of VGG16 under conditions of sufficient ES resources and varying bandwidth. As illustrated in Fig. 4, positioning the partition point immediately after the first convolutional layer consistently yields the minimum end-to-end latency across all bandwidth scenarios. Empirical results show that this partitioning strategy reduces latency by a factor of between 8× (at 100 Mbps) and 21.6× (at 5 Mbps) relative to input-layer partitioning. Crucially, under conditions of sufficient ES resources, this optimal partition point remains invariant to fluctuations in bandwidth.

In addition, experiments were conducted on four classical heterogeneous deep neural network (DNN) architectures and their respective expert models under the conditions of fixed bandwidth and constrained resources. As illustrated in Fig. 5 and Fig. 6, the optimal partition points differ across traditional DNN models. Specifically, AlexNet's optimal partition lies at the L2 layer, while VGG16, ResNet18, and InceptionNet exhibit their optimal partition points at L0 (i.e., full offloading is implemented), and MobileNet's optimal partition is found at the L10 layer.

Furthermore, the L1 layer in traditional DNN models is typically configured as a convolutional layer with a substantial number of convolutional channels. A greater number of convolutional channels leads to a larger volume of intermediate feature data in the output, accompanied by increased redundant data. Consequently, under the premise of preserving inference accuracy, it becomes feasible to reduce redundant features by activating only a subset of channels in the L1 layer thereby improving overall inference latency. As depicted in Figs. 5 and 6, when the expert model architecture is applied to the L1 layer (with only partial convolutional channels activated while ensuring inference accuracy), the minimal overall inference latency can only be achieved by partitioning the model after the L1 layer. Additionally, as presented in Table 2, after integrating the expert model architecture into the L1 layer of traditional DNN models, both transmission latency and overall inference latency exhibit a significant reduction. For this reason, during the optimal partition point selection process, MAE empirically fixes the L1 layer as the optimal partition point and deploys the expert model architecture accordingly. Formally, for any task type $k \in \mathcal{K}$, s_k^* = post-conv1, which eliminates the $O(L)$ layer traversal complexity.

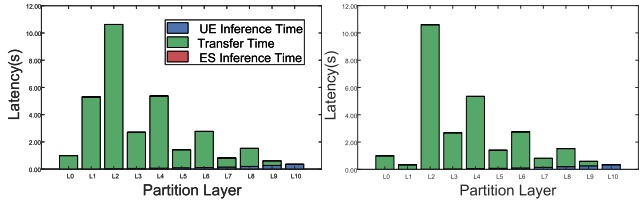
2. Channel-constrained Expert Model Selection: For each task type k, MAE aims to select an expert model $e_k^i \in \mathcal{E}_k$ that minimizes the number of activated convolutional channels while maintaining the inference accuracy of the original DNN:

$$\begin{aligned} \mathbf{P}: \quad & \min \mathcal{CH}(e_k^i) \\ \text{s.t.} \quad & \text{Accuracy}(e_k^i) \geq \text{Accuracy(original model)} \end{aligned} \tag{11}$$

where $\mathcal{CH}(e_k^i)$ denotes the number of activated convolutional channels in expert model e_k^i and Accuracy$(\cdot)$ represents its inference accuracy on the target task. Given the variations in convolutional channels activated by different expert models, the extent of inference accuracy degradation resulting from these differences remains unclear. Consequently, efficient selection of expert models cannot be performed during online inference. To address this challenge, MAE adopts an offline profiling strategy that decouples expert model construction from online inference. Specifically, MAE enumerates candidate expert models offline, profiles their channel activation patterns, and accuracy performance, and stores the resulting mappings in a lookup table. This design enables efficient expert selection at runtime with negligible overhead while preserving accuracy guaranties.

The offline procedure begins by constructing the expert space through a binary search over the channel count c of a designated convolutional layer in the target DNN, where $c \in [c_{\min}, c_{\max}]$. Here, $c_{\min}$ represents a small fraction of the original channel width, and $c_{\max}$ corresponds to the full model. This process efficiently explores the feasible sparsity range and yields a compact yet representative expert set $\mathcal{E}_k$, in which each expert model is characterized by a distinct sparse channel activation configuration. Each candidate expert in $\mathcal{E}_k$ is then fine-tuned from the same pre-trained baseline using the task-specific dataset (e.g., CIFAR-10). The training settings of the model and the corresponding expert model are consistent with those in the evaluation section.

The offline training process constitutes a one-time preprocessing cost for each target DNN. Its computational overhead scales linearly with the number of candidate expert models generated through the binary search procedure. Even for models with relatively large parameter sizes, such as ResNet-18 evaluated in this work, the entire offline training and profiling process can be completed within a few hours using mainstream GPU accelerators. Given the long operational lifetime of edge inference



(a) MobileNet **(b)** MobileNet EM

Fig. 6. Inference Latency Comparison Across Different Partitioning Strategies for MobileNet and Expert Model (EM).

Table 2
A comparative analysis of optimal inference latency between the original model and expert model.

Model	Latency			
	UE	Transfer	ES	Total
AlexNet	31.5	67.0	2.8	**101.3**
AlexNet EM	17.6	8.7	2.5	**28.8**
VGG16	0.0	216.2	35.7	**251.9**
VGG16 EM	5.7	72.1	32.2	**110.0**
ResNet18	0.0	216.2	58.3	**274.5**
ResNet18 EM	6.8	144.1	59.6	**210.5**
InceptionNet	120.2	216.2	16.8	**353.2**
InceptionNet EM	19.5	72.1	18.2	**108.7**
MobileNet	353.0	0.0	0.0	**353.0**
MobileNet EM	2.4	330.9	3.7	**337.0**

systems, this one-time overhead is negligible and can be effectively amortized over a large volume of inference requests, resulting in minimal impact on overall system efficiency.

4.2. Algorithmic transformation

The original problem **P1** is transformed into a tractable optimization problem and can be addressed by the following two steps:

Step 1: Precomputation of Task-type Parameters. Based on Section 4.1, for each task type $k \in \mathcal{K}$, the optimal partition point is fixed (i.e., s_k = post-conv1) and the expert model e_k is determined through quickly searching the lookup table. Thus, MAE computes the following parameters:

$$\tau_k^{\text{mob}} = \frac{1}{n_k} \sum_{i \in \mathcal{N}_k} \frac{X(s_k^i, e_k^i)}{C_i} \tag{12}$$

$$\tau_k^{\text{trans}} = \frac{1}{n_k} \sum_{i \in \mathcal{N}_k} \frac{M(s_k^i, e_k^i)}{B_i} \tag{13}$$

$$\gamma_k = Y(s_k^i, e_k^i) \tag{14}$$

Step 2: Resource Allocation Formulation. The optimization problem reduces to allocating MCRUs $\{f_k\}$ to task types k:

$$\mathbf{P2}: \min D \tag{15a}$$

$$\text{s.t. } \tau_k^{\text{mob}} + \tau_k^{\text{trans}} + \frac{\gamma_k}{f_k \cdot \alpha} \le D_\gamma \tag{15b}$$

$$\sum_{k \in \mathcal{K}} n_k f_k \le \beta \tag{15c}$$

$$f_k \in \mathbb{Z}^+, \forall k \in \mathcal{K} \tag{15d}$$

Note that $f_k \cdot \alpha$ represents the processing power allocated per task of type k.

The proposed MAE resource allocation algorithm is shown in Algorithm 1. Initially, the lower bound D_{low} is set to 0, and the upper bound D_{high} is set to the maximum possible latency incurred when each task is allocated only one MCRU (Line 1–2). The bisection search then iteratively tightens these bounds (Line 3–15). In each iteration, a candidate mid-value D_{mid} is tested for feasibility. For each task type k, the minimum number of MCRUs $f_k^{\min}$ required to meet the latency constraint D_{mid} is computed (Line 7). The total MCRUs required across all task types, $\beta_{\text{req}} = \sum_{k \in \mathcal{K}} n_k \cdot f_k^{\min}$, is then summed. If β_{req} does not exceed the total available MCRUs β, D_{mid} is feasible, and the upper bound D_{high} is reduced. Otherwise, the lower bound D_{low} is increased (Line 13). The loop terminates when the search interval is smaller than a predefined tolerance ϵ, ensuring ϵ-optimality. Finally, the optimal allocation f_k for each task type is derived using the converged value D^*. This algorithm efficiently converges to the optimal solution with a time complexity of $O(|\mathcal{K}| \log(1/\epsilon))$, making it highly scalable for edge environments with limited task-type varieties.

Table 3
Time Complexity Breakdown.

Phase	Traditional	MAE
Partitioning	$O(L)$ per task	$O(1)$
Expert Selection	$O(\lvert\mathcal{E}\rvert)$ per task	$O(1)$
Resource Allocation	$O(N^2)$ or higher	$O(\lvert\mathcal{K}\rvert \log \frac{1}{\epsilon})$
Total	$O(NL + N\lvert\mathcal{E}\rvert + N^2)$	$O(\lvert\mathcal{K}\rvert \log \frac{1}{\epsilon})$

4.3. Complexity analysis

The MAE framework achieves unprecedented efficiency through algorithmic stratification. As shown in Table 3, $|\mathcal{K}|$ is bounded (e.g., 5–10 task types in real systems) and independent of N. Therefore, the overall complexity reduces to $O(\log \frac{1}{\epsilon})$, effectively $O(1)$ for practical purposes. For increasing N with fixed $|\mathcal{K}|$, complexity remains constant.

In Table 3, we define the following notation: L denotes the number of potential partition points, $|\mathcal{E}|$ represents the count of expert models, and $|\mathcal{K}|$ stands for the number of distinct task types (where $|\mathcal{K}| \ll \mathcal{N}$). Additionally, ϵ denotes the precision tolerance, which is typically set to 10^{-3}.

Algorithm 1 MAE Resource Allocation.

Require: Task types $\mathcal{K}$, counts $\{n_k\}$, parameters $\{\tau_k^{\text{mob}}, \tau_k^{\text{trans}}, \gamma_k\}$, total MCRUs β
Ensure: Resource allocation $\{f_k\}$, min-max latency D^*
1: $D_{\text{low}} \leftarrow 0$ ▷ Lower bound
2: $D_{\text{high}} \leftarrow \max_k \left(\tau_k^{\text{mob}} + \tau_k^{\text{trans}} + \frac{\gamma_k}{\alpha}\right)$ ▷ Upper bound
3: **while** $D_{\text{high}} - D_{\text{low}} > \epsilon$ **do**
4: $D_{\text{mid}} \leftarrow (D_{\text{low}} + D_{\text{high}})/2$
5: $\beta_{\text{req}} \leftarrow 0$ ▷ Total MCRUs required for D_{mid}
6: **for** each task type $k \in \mathcal{K}$ **do**
7: $f_k^{\min} \leftarrow \left\lceil \frac{\gamma_k}{\alpha(D_{\text{mid}} - \tau_k^{\text{mob}} - \tau_k^{\text{trans}})} \right\rceil$
8: $p_k \leftarrow f_k^{\min} \cdot \alpha$ ▷ Required processing power
9: **end for**
10: **if** $\beta_{\text{req}} \le \beta$ **then**
11: $D_{\text{high}} \leftarrow D_{\text{mid}}$ ▷ Feasible latency
12: **else**
13: $D_{\text{low}} \leftarrow D_{\text{mid}}$ ▷ Infeasible latency
14: **end if**
15: **end while**
16: $D^* \leftarrow D_{\text{high}}$
17: Compute $\{f_k\}$ using D^* (Line 7)

4.4. Optimality guarantee

This subsection presents rigorous proofs establishing the convergence and bounded-error properties of the proposed algorithm.

Lemma 1 (Upper Bound Initialization Validity). *For any resource allocation scheme satisfying $\sum_i f_i \le \beta$, the maximum latency D is bounded by:*

$$D \le \max_{k\in\mathcal{K}} \left(\tau_k^{\text{mob}} + \tau_k^{\text{trans}} + \frac{\gamma_k}{\alpha}\right) \triangleq D_{\max} \tag{16}$$

Proof. Consider the minimal resource allocation where each task receives exactly one MCRU ($f_k = 1$). The edge computation latency for task type k satisfies:

$$t_k^{\text{edge}} = \frac{\gamma_k}{f_k \cdot \alpha} \le \frac{\gamma_k}{\alpha} \tag{17}$$

since $f_k \ge 1$. Therefore, by combining this with the UEs computation and intermediate feature transmission latencies, we have:

$$\begin{aligned} t_k &\le \tau_k^{\text{mob}} + \tau_k^{\text{trans}} + \frac{\gamma_k}{\alpha} \\ &\le \max_{k\in\mathcal{K}} \left(\tau_k^{\text{mob}} + \tau_k^{\text{trans}} + \frac{\gamma_k}{\alpha}\right) = D_{\max} \end{aligned} \tag{18}$$

Thus $D = \max(t_k) \le D_{\max}$ for any feasible allocation. □

Theorem 1 (ϵ-Optimality). *Given a precision tolerance $\epsilon > 0$, Algorithm 1 outputs a minimal maximum end-to-end latency D^* such that:*

$$|D^* - D^{\text{opt}}| \le \epsilon \tag{19}$$

where D^{opt} denotes the true minimal maximum end-to-end latency, with time complexity:

$$\mathcal{O}\left(|\mathcal{K}| \log_2\left(\frac{D_{\max} - D_{\min}}{\epsilon}\right)\right) \tag{20}$$

Theorem 2. *Let $F(D)$ denote the feasibility function, which returns 1 if the latency constraint D can be satisfied with the available MCRUs β, and 0 otherwise. We establish convergence via three properties: 1. **Monotonicity**: The feasibility function $\mathcal{F}(D)$ is monotonically non-increasing in D. For any $D_1 < D_2$:*

$$\mathcal{F}(D_1) = 1 \Rightarrow \mathcal{F}(D_2) = 1 \tag{21}$$

since reduced latency constraints relax the optimization problem.

*2. **Bounded Search Space**: By Lemma 1, $D_{\text{high}}^0 = D_{\max}$ is a valid upper bound. The lower bound $D_{\text{low}}^0 = 0$ is trivial since $t_k > 0$.*

*3. **Linear Convergence**: Each bisection step halves the search interval:*

$$\Delta^{(n)} = \frac{D_{\text{high}}^0 - D_{\text{low}}^0}{2^n} \tag{22}$$

where $\Delta^{(n)}$ denotes the search interval length after n iterations. Termination occurs when $\Delta^{(n)} \le \epsilon$, requiring at most $n_{\max} = \left\lceil \log_2\left(\frac{D_{\max}-0}{\epsilon}\right)\right\rceil$ iterations.

At termination, $D^* = D_{\text{high}}$ satisfies $D^{\text{opt}} \le D^* \le D^{\text{opt}} + \epsilon$ since D^* is feasible ($\mathcal{F}(D^*) = 1$) and $D^* - \epsilon/2$ was infeasible in the previous iteration. The per-iteration cost is $\mathcal{O}(|\mathcal{K}|)$ from the resource requirement computation. Thus, the total complexity is $\mathcal{O}(|\mathcal{K}| \log_2(D_{\max}/\epsilon))$.

Theorem 3 (Deployment Error Bound). *The practical deployment latency $\hat{D}$ exceeds the predicted optimal latency D^* by at most:*

$$\hat{D} - D^* \le \max_{k\in\mathcal{K}} \left(\delta_k^{\text{mob}} + \delta_k^{\text{trans}} + \frac{\gamma_k(\eta-1)}{\alpha}\right) \tag{23}$$

where:

- *δ_k^{mob} is the upper bound for UEs computation time measurement error,*
- *δ_k^{trans} is the upper bound for transmission time estimation error, and*
- *$\eta \ge 1$ is the containerization overhead factor (empirically $\eta \le 1.05$).*

Proof. Let $\hat{t}_k$ denote actual latency for task type k:

$$\begin{aligned} \hat{t}_k &= \underbrace{(\tau_k^{\text{mob}} + \Delta\tau_k^{\text{mob}})}_{\text{actual UEs}} + \underbrace{(\tau_k^{\text{trans}} + \Delta\tau_k^{\text{trans}})}_{\text{actual trans}} + \frac{\gamma_k}{p\_k/\eta} \\ &\le (\tau_k^{\text{mob}} + \delta_k^{\text{mob}}) + (\tau_k^{\text{trans}} + \delta_k^{\text{trans}}) + \frac{\gamma_k \eta}{f_k \alpha} \end{aligned} \tag{24}$$

Since f_k satisfies the constraint at D^*:

$$\tau_k^{\text{mob}} + \tau_k^{\text{trans}} + \frac{\gamma_k}{f_k \alpha} \le D^* \tag{25}$$

Combining inequalities:

$$\hat{t}_k \le D^* + \delta_k^{\text{mob}} + \delta_k^{\text{trans}} + \frac{\gamma_k(\eta-1)}{f_k\alpha} \tag{26}$$

Using $f_k \ge 1$ and $\gamma_k/(f_k\alpha) \le D^* \le D_{\max}$:

$$\hat{D} = \max_k \hat{t}_k \le D^* + \max_k \left(\delta_k^{\text{mob}} + \delta_k^{\text{trans}} + \frac{\gamma_k(\eta-1)}{\alpha}\right) \tag{27}$$

Empirical measurements show $\delta_k^{\text{mob}}, \delta_k^{\text{trans}} < 1$ ms and $\eta = 1.05$ for Docker containers. □

5. Experimental evaluation

5.1. Experimental setup

5.1.1. Hardware configuration

To evaluate system performance under realistic conditions, we deployed a heterogeneous edge computing testbed comprising a central ES and multiple user terminals. ES is equipped with an NVIDIA GeForce GTX 1660 Ti GPU as the primary accelerator for convolutional workloads, along with dual Intel Xeon Silver 4314 processors and 256 GB of RAM to support concurrent processing. Each user terminal provides local computational capability through a CPU with 16 GB of RAM and emulated 4G/5G cellular connectivity, simulating real-world network conditions. At the server side, we leverage Docker containers to isolate computational resources for each user device. To faithfully emulate the resource-constrained edge computing scenarios central to our study, we configured the ES with a deliberately limited pool of computational resources, quantified as MCRUs as defined in our system model (Section 3). The total number of available MCRUs was set to $\beta = 200$, with each MCRU delivering $\alpha = 50$ GFLOPS/s. This setting is designed to create resource contention under the baseline workload of 6 AlexNet and 6 VGG16 tasks.

5.1.2. Software & workloads

For our experimental evaluation, we employed PyTorch 1.12 with CUDA 11.6 as the deep learning framework to leverage GPU acceleration capabilities. The benchmark workloads utilized the CIFAR-10 dataset [18] comprising 60,000 32×32 RGB images across 10 object classes. To represent diverse edge inference scenarios, six instances of AlexNet were deployed as lightweight models (approximately 61 million FLOPs per instance) alongside six instances of VGG-16 as computationally intensive models (approximately 15.5 billion FLOPs per instance) across different UEs. To ensure fair comparison and isolate the impact of channel sparsity, all baseline models and their corresponding expert models were trained using identical hyperparameters: SGD with a momentum of 0.9, weight decay of 5×10^{-4}, and a step-wise learning rate decay applied at 50% and 75% of the total training epochs. AlexNet and VGG-16 architectures were trained with an initial learning rate of 0.01 and 0.05, and a batch size of 128 and 64. The accuracy of all models was evaluated on a pre-defined test set, which comprised a randomly selected 20% subset of the CIFAR-10 dataset, partitioned prior to the commencement of training.

5.1.3. Comparison methods

We evaluated the proposed MAE framework against the following four baseline strategies: (1) Local Execution, where inference tasks are executed entirely on UEs without offloading to the ES. This strategy does not involve resource allocation on the ES and relies solely on the terminal's own computational capability. (2) Full Offloading refers to the approach where UEs transmit complete inference tasks to the ES for execution. Under this strategy, multiple terminals compete for limited edge computing resources and newly arrived tasks must wait until ongoing tasks finish and resources are released if allocation is full.

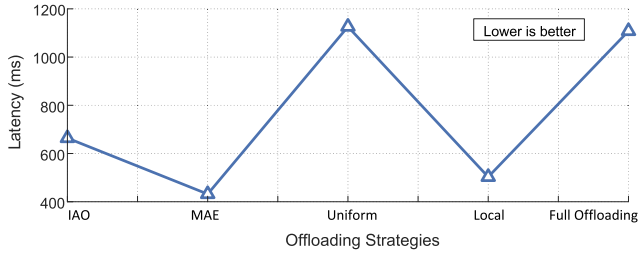


Fig. 7. Comparison of different strategies under low-bandwidth scenarios (5 Mbps).

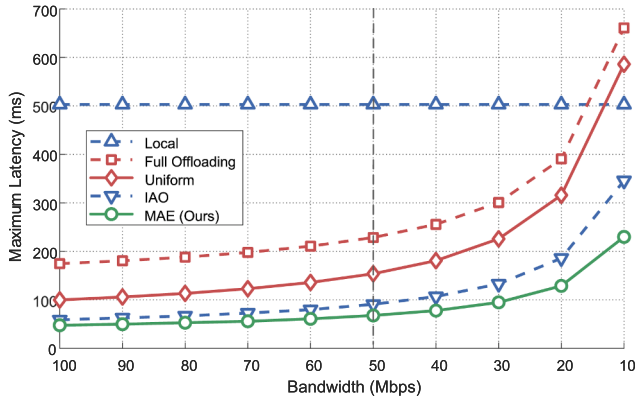


Fig. 8. Comparison of different strategies under high-bandwidth scenarios.

(3) Uniform Allocation [19,36], which first traverses all layers of the model to determine the optimal partition point by comparing end-to-end inference latency across different partition locations. All tasks are offloaded according to this optimal partition point and are allocated an equal amount of computational resources on the ES. (4) IAO (Iterative Alternating Optimization) [20], which jointly determines the optimal partition point and performs dynamic resource allocation based on an iterative alternating optimization method, with the goal of minimizing overall inference latency. These baseline methods provide representative references for device-edge collaborative inference from different perspectives.

5.2. MAE efficacy under diverse bandwidth

We conducted extensive experiments across diverse bandwidth conditions to validate MAE efficacy. As shown in Figs. 7 and 8, our framework consistently outperforms all baselines in both high and low-bandwidth conditions. Specifically, under constrained 5 Mbps network conditions (Fig. 7), MAE achieves a latency reduction of 35% compared to local execution and a 61% performance improvement over full offloading. This superiority originates from the adopted expert model selection, which reduces the transmission volume by a factor of 4.8 relative to conventional offloading approaches. As the bandwidth increases to 100 Mbps, MAE retains a 19% performance advantage over IAO. This is accomplished by dynamically adjusting resource allocation strategies to minimize computational bottlenecks.

As shown in Fig. 8, the proposed MAE framework consistently outperforms all baseline methods under high-bandwidth conditions. More notably, it exhibits even more significant advantages in lower bandwidth scenarios. Specifically, at a bandwidth of 20 Mbps, MAE achieves a 33.5% reduction in latency compared to IAO (the best-performing baseline algorithm). This improvement can be attributed to the adoption of the expert model selection, which substantially reduces the volume of intermediate data during collaborative DNN inference, thereby significantly reducing the end-to-end inference latency.

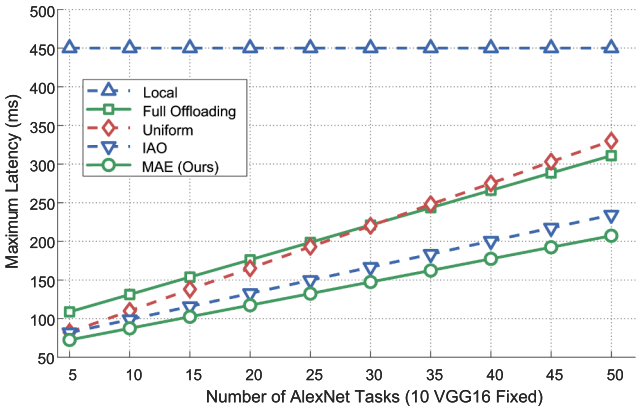


Fig. 9. Scalability analysis with increasing AlexNet tasks.

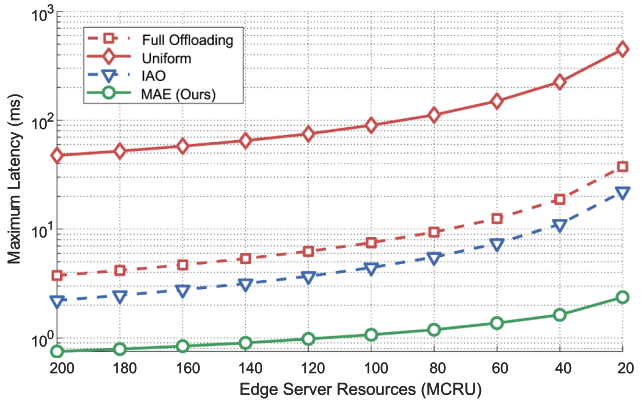


Fig. 10. Performance under resource constraints.

5.3. Scalability analysis under heterogeneous workloads

To rigorously evaluate the scalability of MAE under heterogeneous task distributions, we designed and conducted two complementary experiments under fixed bandwidth (100 Mbps)). These experiments collectively validate MAE's robustness against both workload scaling and resource constraints, providing comprehensive evidence of its adaptive performance in dynamic environments.

Fig. 9 presents latency measurements obtained under a scenario where the number of AlexNet tasks increases from 5 to 50, while the number of VGG16 tasks is maintained at 10. The experimental data reveals several critical insights: Compared to IAO, the proposed MAE framework demonstrates consistent superiority as the task scale expands. When the total number of tasks grows tenfold, the DNN inference latency of MAE increases from 72.5 ms to 207.2 ms; notably, this translates to an average performance improvement of 13% over IAO. The logarithmic scaling enables MAE to maintain stable performance under heavy loads.

Fig. 10 presents system performance evaluations under severe edge resource constraints, where edge resources are reduced from 200 to 20 MCRUs. Critical findings contain (1)Graceful degradation: MAE exhibits superior robustness, as its latency increases by merely 316% (from 0.75 ms to 2.37 ms) when edge resources are reduced by 90%. This performance significantly outperforms that of Full Offload (1000% latency increase), Uniform Allocation (947% latency increase), and IAO (1000% latency increase). (2) Critical threshold behavior: When edge resources drop below 40 MCRUs (20% of the initial resource amount), MAE's advantage is further amplified at 40 MCRUs. At this level, it achieves 6.5× lower latency than IAO (1.63 ms for MAE vs. 10.6 ms for IAO) and 138× lower latency than Full Offload (1.63 ms for MAE vs. 225 ms for Full Offload). (3) Resource-efficient design: MAE maintains a sub-2.5 ms

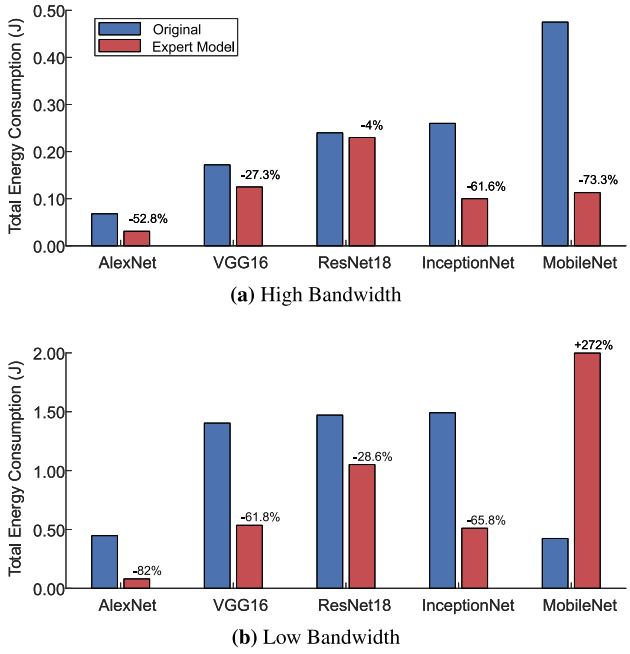


Fig. 11. Energy consumption under High and Low Bandwidth.

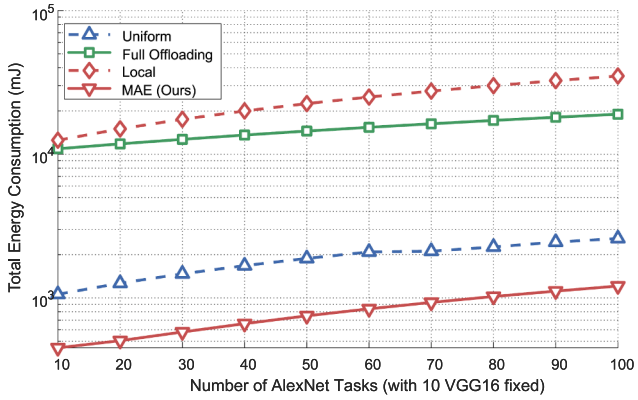


Fig. 12. Scalability analysis with increasing AlexNet tasks for energy comparison.

end-to-end latency even at 20 MCRUs. This is attributed to its minimal overhead of expert selection and optimal partitioning strategy, which reduce resource demands by a factor of 4.8 compared to conventional offloading approaches.

5.4. Energy consumption

The energy consumption of an inference task comprises three components: UEs computation energy, transmission energy, and edge computation energy. UEs computation energy is defined as the product of the UE's average power during DNN execution and its local inference time. Transmission energy equals the UE's transmission power multiplied by the duration of intermediate feature transmission. Edge computation energy scales linearly with the allocated computational resources (MCRUs) and the server's power consumption. The total task energy is the sum of these components. In multi-task scenarios, system-wide energy consumption is obtained by aggregating energy usage across all UEs.

Fig. 11 illustrates the total energy consumption corresponding for each type of DNN models under high-bandwidth (100 Mbps) and low-bandwidth (5 Mbps) settings. Overall, the expert models in the MAE framework consistently achieve substantial energy savings. Under high-bandwidth condition, reduced transmission latency diminishes the impact of transmission energy. Even so, by activating only a sparse subset of convolutional channels, expert models significantly decrease intermediate feature sizes, improving both UEs and edge computation efficiency. Compared with their baseline models, expert models achieve energy reductions ranging from 4.0% (ResNet18 EM) to 73.3% (MobileNet EM). In low-bandwidth scenario, transmission energy dominates due to extended data transfer time. Here, the benefits of EMs become more pronounced: aggressive feature compression leads to energy savings between 28.6% (ResNet18 EM) and 82.0% (AlexNet EM). These results confirm that the MAE framework delivers superior energy efficiency, particularly in bandwidth-constrained edge inference environments.

A notable exception arises for MobileNet. Under low-bandwidth condition, the expert model (MobileNet EM) consumes approximately 272% more energy than executing the original MobileNet entirely on the mobile device. This counterintuitive behavior can be attributed to two factors. First, in bandwidth-constrained environments, collaborative inference incurs substantial transmission overhead for intermediate features. Second, MobileNet is an lightweight model with inherently low local computation energy. Consequently, for such model, purely local execution is more energy-efficient than collaborative inference under limited bandwidth.

To further investigate the energy consumption characteristics of MAE in scalable heterogeneous task scenarios, we conducted dedicated experiments under a fixed bandwidth condition of 100 Mbps. Fig. 12 further demonstrates that the MAE framework consistently achieves the lowest total energy consumption across all task configurations. As the number of AlexNet inference tasks increases from 10 to 100, MAE exhibits a logarithmic growth trend, with total energy rising from 509 mJ to 1209 mJ. Compared with uniform partitioning, MAE reduces energy consumption by 53.5%, and achieves orders-of-magnitude savings relative to full offloading and fully local execution.

6. Conclusion

In this paper, we propose a novel Mixture of Adaptive Experts (MAE) framework to address the efficiency bottleneck of DNN inference in heterogeneous MEC environments. By reconfiguring convolutional channels into specialized "experts" that are sparsely activated during each inference process, MAE effectively reduces the transmission overhead of intermediate features while preserving model accuracy. This approach enables efficient model partitioning and subsequent selection of expert models. Moreover, MAE reformulates the computationally intractable resource scheduling problem into an optimization problem characterized by a monotonically decreasing objective function. To solve this problem, we develop the MAE Resource Allocation algorithm, which is capable of identifying the optimal solution with logarithmic time complexity. To evaluate the performance of the MAE framework, we conduct extensive experiments on real-world heterogeneous DNN tasks. The results demonstrate that our solution significantly improves overall system performance in terms of inference efficiency while maintaining inference accuracy, confirming its practical effectiveness and efficiency in edge computing scenarios.

The current MAE framework is based on convolutional channel reconstruction, treating individual channels as specialized experts. While effective for CNNs, this design limits direct applicability to non-convolutional architectures such as Vision Transformers, which rely on self-attention rather than channel-wise representations. Future work will extend MAE to a broader range of DNN architectures, including federated learning settings, and explore more general expert-activation mechanisms compatible with diverse network backbones. We also plan to evaluate MAE on additional vision tasks beyond classification, such as semantic segmentation. We also plan to incorporate energy-aware scheduling for battery-constrained IoT deployments.

CRediT authorship contribution statement

Juan Fang: Writing – original draft, Supervision, Methodology; **Yaxin An:** Investigation, Software, Validation, Methodology; **Yaqi Liu:** Writing – original draft, Validation, Methodology; **Ziyi Teng:** Software, Validation, Visualization; **Xiaoning Zhai:** Software, Data curation; **Heng Tang:** Software, Validation; **Huijie Chen:** Project administration, Funding acquisition, Writing – original draft, Methodology.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by National Natural Science Foundation of China (62202019, 61202076), and supported by the Beijing Municipal Natural Science Foundation under Grant 4262021, along with other government sponsors. Corresponding author is Huijie Chen.

References

[1] Y. Hu, Q. Jia, Y. Yao, Y. Lee, M. Lee, C. Wang, X. Zhou, R. Xie, F.R. Yu, Industrial internet of things intelligence empowering smart manufacturing: a literature review, IEEE Internet Things J. 11 (11) (2024) 19143–19167.
[2] T. Gong, L. Zhu, F.R. Yu, T. Tang, Edge intelligence in intelligent transportation systems: a survey, IEEE Trans. Intell. Transp. Syst. 24 (9) (2023) 8919–8944.
[3] C. Chen, C. Wang, B. Liu, C. He, L. Cong, S. Wan, Edge intelligence empowered vehicle detection and image segmentation for autonomous vehicles, IEEE Trans. Intell. Transp. Syst. 24 (11) (2023) 13023–13034.
[4] H. Hua, Y. Li, T. Wang, N. Dong, W. Li, J. Cao, Edge computing with artificial intelligence: a machine learning perspective, ACM Comput. Surv. 55 (9) (2023) 1–35.
[5] J. Fang, D. Qu, H. Chen, Y. Liu, Dependency-aware dynamic task offloading based on deep reinforcement learning in mobile-edge computing, IEEE Trans. Netw. Serv. Manage. 21 (2) (2023) 1403–1415.
[6] J.H. Syu, J.C.W. Lin, G. Srivastava, K. Yu, A comprehensive survey on artificial intelligence empowered edge computing on consumer electronics, IEEE Trans. Consum. Electron. 69 (4) (2023) 1023–1034.
[7] S. Zhang, N. Yi, Y. Ma, A survey of computation offloading with task types, IEEE Trans. Intell. Transp. Syst. 25 (8) (2024) 8313–8333.
[8] A. Younis, S. Maheshwari, D. Pompili, Energy-latency computation offloading and approximate computing in mobile-edge computing networks, IEEE Trans. Netw. Serv. Manage. 21 (3) (2024) 3401–3415.
[9] H. Cheng, M. Zhang, J.Q. Shi, A survey on deep neural network pruning: taxonomy, comparison, analysis, and recommendations, IEEE Trans. Pattern Anal. Mach. Intell. 46 (12) (2024) 10558–10578.
[10] B.J. Eccles, P. Rodgers, P. Kilpatrick, I. Spence, B. Varghese, DNNShifter: An efficient DNN pruning system for edge computing, Future Gener. Comput. Syst. 152 (2024) 43–54.
[11] M. Sepahvand, F. Abdali-Mohammadi, A. Taherkordi, An adaptive teacher-student learning algorithm with decomposed knowledge distillation for on-edge intelligence, Eng. Appl. Artif. Intell. 117 (2023) 105560.
[12] Y. Kang, J. Hauswald, C. Gao, et al, Neurosurgeon: collaborative intelligence between the cloud and mobile edge, ACM SIGARCH Computer Architecture News 45 (1) (2017) 615–629.
[13] Y. Duan, J. Wu, Optimizing job offloading schedule for collaborative DNN inference, IEEE Trans. Mob. Comput. 23 (4) (2023) 3436–3451.
[14] S.K. Ghosh, A. Raha, V. Raghunathan, A. Raghunathan, Partnner: platform-agnostic adaptive edge-cloud DNN partitioning for minimizing end-to-end latency, ACM Trans. Embedded Comput. Syst. 23 (1) (2024) 1–38.
[15] H. Li, X. Li, Q. Fan, Q. He, X. Wang, V.C.M. Leung, Distributed DNN inference with fine-grained model partitioning in mobile edge computing networks, IEEE Trans. Mob. Comput. 23 (10) (2024) 9060–9074.
[16] P. Dai, B. Han, K. Li, X. Xu, H. Xing, K. Liu, Joint optimization of device placement and model partitioning for cooperative DNN inference in heterogeneous edge computing, IEEE Trans. Mob. Comput. 24 (1) (2024) 210–226.
[17] R.P. H, V. Srivastava, K. Chaurasia, R.G. Pacheco, R.S. Couto, Early-exit deep neural network-a comprehensive survey, ACM Comput. Surv. 57 (3) (2024) 1–37.
[18] D. Bankman, L. Yang, B. Moons, M. Verhelst, B. Murmann, An always-on 3.8μj/86binary cnn processor with all memory on chip in 28-nm cmos, IEEE J. Solid State Circuits 54 (1) (2018) 158–172.
[19] Q. Wu, W. Wang, P. Fan, Q. Fan, J. Wang, K.B. Letaief, Urllc-awared resource allocation for heterogeneous vehicular edge computing, IEEE Trans. Veh. Technol. 73 (8) (2024) 11789–11805.
[20] X. Tang, X. Chen, L. Zeng, S. Yu, L. Chen, Joint multiuser DNN partitioning and computational resource allocation for collaborative edge intelligence, IEEE Internet Things J. 8 (12) (2021) 9511–9522.
[21] W. Fan, L. Gao, Y. Su, F. Wu, Y. Liu, Joint DNN partition and resource allocation for task offloading in edge-cloud-assisted iot environments, IEEE Internet Things J. 10 (12) (2023) 10146–10159.
[22] Z. Zhou, M. Shojafar, J. Abawajy, A.K. Bashir, IADE: An improved differential evolution algorithm to preserve sustainability in a 6g network, IEEE Transactions on Green Communications and Networking 5 (4) (2021) 1747–1760.
[23] C. Li, Y. Su, Z. Zhou, Z. Zhao, Y. Wen, J. Chen, J. Wei, Multiagent reinforcement learning-based cost-efficient edge server deployment in CPSS, 2025, pp. 1–13.
[24] Z. Zhou, M. Shojafar, M. Alazab, J. Abawajy, F. Li, AFED-EF: An energy-efficient vm allocation algorithm for iot applications in a cloud data center, IEEE Transactions on Green Communications and Networking 5 (2) (2021) 658–669.
[25] Z. Zhou, J. Abawajy, Reinforcement learning-based edge server placement in the intelligent internet of vehicles environment, 2025, pp. 1–11.
[26] Z. Zhou, M. Shojafar, J. Abawajy, H. Yin, H. Lu, ECMS: An edge intelligent energy efficient model in mobile edge computing, IEEE Transactions on Green Communications and Networking 6 (1) (2021) 238–247.
[27] Z. Zhou, M. Shojafar, M. Alazab, F. Li, IECL: an intelligent energy consumption model for cloud manufacturing, IEEE Trans. Ind. Inf. 18 (12) (2022) 8967–8976.
[28] M. Chen, A. Liu, N.N. Xiong, Y. Ren, H.H. Song, Anns: an intelligent advanced non-convex non-smooth scheme for irs-aided next generation mobile communication networks, IEEE Trans. Mob. Comput. 24 (9) (2025) 8959–8973.
[29] M. Chen, A. Liu, N.N. Xiong, H. Song, V.C. Leung, SGPL: An intelligent game-based secure collaborative communication scheme for metaverse over 5g and beyond networks, IEEE J. Sel. Areas Commun. 42 (3) (2023) 767–782.
[30] M. Chen, A. Liu, W. Liu, K. Ota, M. Dong, N.N. Xiong, RDRL: A recurrent deep reinforcement learning scheme for dynamic spectrum access in reconfigurable wireless networks, IEEE Trans. Network Sci. Eng. 9 (2) (2021) 364–376.
[31] A. Liu, B. Feng, B. Xue, Deepseek-v3 technical report, Technical Report, arXiv preprint, 2024.
[32] R. Kong, Y. Li, W. Wang, L. Kong, Y. Liu, Serving moe models on resource-constrained edge devices via dynamic expert swapping, IEEE Trans. Comput. 74 (8) (2025) 2799–2811.
[33] J. Wang, H. Du, D. Niyato, J. Kang, Z. Xiong, D.I. Kim, K.B. Letaief, Toward scalable generative ai via mixture of experts in mobile edge networks, IEEE Wireless Commun. 32 (1) (2024) 142–149.
[34] H. Li, L. Duan, Theory of mixture-of-experts for mobile edge computing, in: IEEE Conference on Computer Communications (IEEE INFOCOM), IEEE, 2025, pp. 1–10.
[35] M. Zhang, J. Fang, Z. Teng, et al, Joint DNN partitioning and task offloading based on attention mechanism-aided reinforcement learning, IEEE Trans. Netw. Serv. Manage. 22 (3) (2025) 2914–2927.
[36] H. Djigal, J. Xu, L. Liu, Y. Zhang, Machine and deep learning for resource allocation in multi-access edge computing: a survey, IEEE Commun. Surv. Tut. 24 (4) (2022) 2449–2494.

Juan Fang is a professor in the College of Computer Science, Beijing University of Technology (BJUT). She received the MS degree from Jilin University of Technology, Changchun, China in 1997 and her PhD degree from the College of Computer Science, Beijing University of Technology, Beijing, China in 2005. Her research interests include high performance computing, intelligent computing, and edge intelligence.

Yaxin An received his BS degree from Hebei GEO University, Shijiazhuang, China in 2023. Currently, he is working toward a MS degree in Computer Technology under the College of Computer Science at Beijing University of Technology, Beijing, China. He is a student member of CCF.

Yaqi Liu received the MS degree from Beijing University of Technology, Beijing, China in 2022. She is currently pursuing the PhD degree at Beijing University of Technology. Her research interest is mobile edge computing. She is a student member of CCF.

Ziyi Teng received the BS degree from the North China University of Water Resources and Electric Power, Zhengzhou, China, in 2019. She is currently pursuing the PhD degree with the Beijing University of Technology, Beijing, China. Her research interests in mobile edge computing.

Xiaoning Zhai received his BSc in Computer Science and Technology at Yanshan University. Cur rently, he is working toward a MSc degree in Electronic Information under the College of Com puter Science Technology at Beijing University of Technology.

Heng Tang received his BS degree from Tianjin University of Science and Technology, Tianjin, China in 2023. Currently, he is working toward a MS degree in Computer Technology under the College of Computer Science at Beijing University of Technology, Beijing, China.

Huijie Chen received the PhD degree in computer sience from the Beijing Institute of Technology, Beijing, China in 2020. He currently works in the school of computer science, Beijing University of Technology, Beijing, China. His research interests include Smart Sensing, and mobile edge computing.